THE COMPLETE SOLUTIONS MANUAL

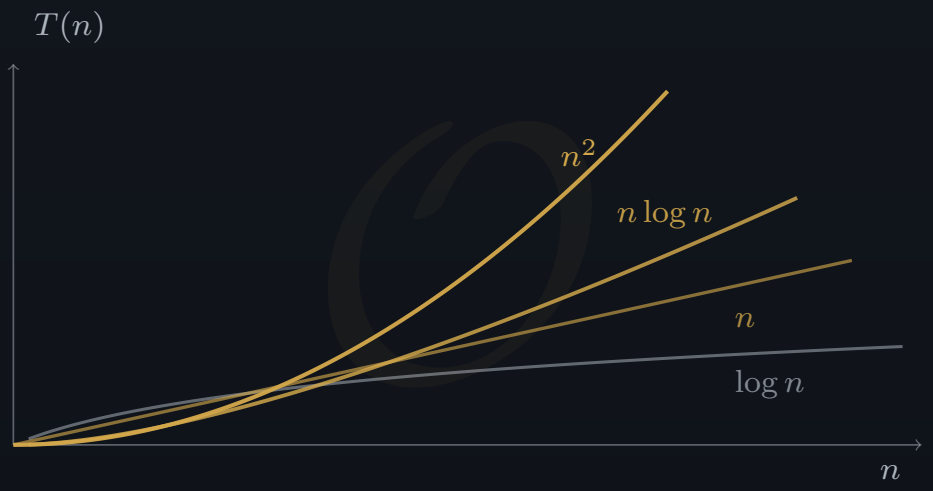
Data Structures & Algorithm Analysis in C++

by Mark Allen Weiss

◇

Chapter 2

Algorithm Analysis



Ahmed Amine Azzouzi

2026

Contents

Exercises **1**

Exercise 2.1: Ordering asymptotic growth rates . . . 1
Exercise 2.2: Properties of Big-Oh bounds . . . 1
Exercise 2.3: Comparing two growth rates . . . 2
Exercise 2.4: Logarithms versus linear growth . . . 2
Exercise 2.5: Incomparable asymptotic functions . . . 2
Exercise 2.6: A repeatedly squared fine . . . 3
Exercise 2.7: Analyzing nested-loop fragments . . . 4
Exercise 2.8: Random permutation algorithms . . . 8
Exercise 2.9: Extrapolating algorithm runtimes . . . 11
Exercise 2.10: Hand arithmetic complexity . . . 12
Exercise 2.11: Scaling runtime with input size . . . 13
Exercise 2.12: Problem size under a time limit . . . 14
Exercise 2.13: Polynomial evaluation by powers . . . 15
Exercise 2.14: Horner's rule . . . 15
Exercise 2.15: Finding a fixed point in an array . . . 16
Exercise 2.16: The binary GCD algorithm . . . 17
Exercise 2.17: Subsequence sums and products . . . 18
Exercise 2.18: Root finding by bisection . . . 20
Exercise 2.19: Returning subsequence endpoints . . . 20
Exercise 2.20: Trial division and input bits . . . 21
Exercise 2.21: Sieve of Eratosthenes complexity . . . 22
Exercise 2.22: Computing a power efficiently . . . 23
Exercise 2.23: Iterative fast exponentiation . . . 23
Exercise 2.24: Counting exponentiation multiplications . . . 24
Exercise 2.25: Comparing runtime guarantees . . . 25
Exercise 2.26: Finding a majority element . . . 25
Exercise 2.27: Searching a sorted matrix . . . 27
Exercise 2.28: Optimizing ordered array pairs . . . 28
Exercise 2.29: Fixed-size integer assumptions . . . 29
Exercise 2.30: Word-puzzle search complexity . . . 30
Exercise 2.31: A binary-search termination bug . . . 31
Exercise 2.32: Binary search with one comparison . . . 31
Exercise 2.33: Incorrect recursive partitioning . . . 32
Exercise 2.34: Counting nested-loop executions . . . 33

Exercises

Exercise 2.1

Order the following functions by growth rate: N, $\sqrt{N}$, $N^{1.5}$, N^2, $N \log N$, $N \log \log N$, $N \log^2 N$, $N \log(N^2)$, $2/N$, 2^N, $2^{N/2}$, 37, $N^2 \log N$, and N^3. Indicate which functions grow at the same rate.

Solution

As $N \to \infty$, the functions are ordered from slowest to fastest as follows, where $f \prec g$ means that $f = o(g)$, and $f \asymp g$ means that $f = \Theta(g)$:

$$\begin{aligned}\frac{2}{N} &\prec 37 \prec \sqrt{N} \prec N \prec N \log \log N \\ &\prec N \log N \asymp N \log(N^2) \prec N(\log N)^2 \prec N^{1.5} \\ &\prec N^2 \prec N^2 \log N \prec N^3 \prec 2^{N/2} \prec 2^N.\end{aligned}$$

The only functions in the list that grow at the same asymptotic rate are $N \log N$ and $N \log(N^2)$, since

$$N \log(N^2) = 2N \log N = \Theta(N \log N).$$

Exercise 2.2

Suppose $T_1(N) = O(f(N))$ and $T_2(N) = O(f(N))$. Which of the following are true?

a. $T_1(N) + T_2(N) = O(f(N))$

b. $T_1(N) - T_2(N) = o(f(N))$

c. $\dfrac{T_1(N)}{T_2(N)} = O(1)$

d. $T_1(N) = O(T_2(N))$

Solution

a. **True.** If $T_1(N) \leq c_1 f(N)$ and $T_2(N) \leq c_2 f(N)$ for sufficiently large N, then

$$T_1(N) + T_2(N) \leq (c_1 + c_2) f(N).$$

b. **False.** For example, let $T_1(N) = 2f(N)$ and $T_2(N) = f(N)$. Their difference is $f(N)$, which is not $o(f(N))$.

c. **False.** Both functions can be bounded above by $f(N)$ while having very different sizes. For example, if $f(N) = N$, $T_1(N) = N$, and $T_2(N) = 1$, then $T_1(N)/T_2(N) = N$, which is not $O(1)$.

d. **False.** Using the same example, $T_1(N) = N$ is not $O(T_2(N)) = O(1)$.

Therefore, only statement **(a)** is necessarily true.

Exercise 2.3

Which function grows faster: $N \log N$ or $N^{1+\varepsilon/\sqrt{\log N}}$, $\varepsilon > 0$?

Solution

Consider their ratio:

$$\frac{N^{1+\varepsilon/\sqrt{\log N}}}{N \log N} = \frac{N^{\varepsilon/\sqrt{\log N}}}{\log N}.$$

Using natural logarithms (a different fixed base changes only a positive constant), this becomes

$$\frac{e^{\varepsilon\sqrt{\log N}}}{\log N}.$$

Its logarithm is

$$\varepsilon\sqrt{\log N} - \log\log N,$$

which tends to infinity because $\sqrt{\log N}$ grows faster than $\log\log N$. Therefore,

$$N^{1+\varepsilon/\sqrt{\log N}} = \omega(N \log N),$$

so $N^{1+\varepsilon/\sqrt{\log N}}$ grows faster.

Exercise 2.4

Prove that for any constant k, $\log^k N = o(N)$.

Solution

Let $x = \log N$. The base of the logarithm affects only a constant factor, so we may use the natural logarithm. Then $N = e^x$, and

$$\frac{\log^k N}{N} = \frac{x^k}{e^x}.$$

Choose an integer $m > k$. From the power-series expansion of e^x, for $x > 0$ we have

$$e^x \geq \frac{x^m}{m!}.$$

Consequently,

$$0 \leq \frac{x^k}{e^x} \leq \frac{m!}{x^{m-k}} \longrightarrow 0 \qquad \text{as } x \to \infty.$$

Therefore,

$$\lim_{N\to\infty} \frac{\log^k N}{N} = 0,$$

which proves that $\log^k N = o(N)$ for every fixed constant k.

Exercise 2.5

Find two functions $f(N)$ and $g(N)$ such that neither $f(N) = O(g(N))$ nor $g(N) = O(f(N))$.

Solution

Define

$$f(N) = \begin{cases} N, & \text{if } N \text{ is even,} \\ N^2, & \text{if } N \text{ is odd,} \end{cases} \qquad g(N) = \begin{cases} N^2, & \text{if } N \text{ is even,} \\ N, & \text{if } N \text{ is odd.} \end{cases}$$

For odd N,

$$\frac{f(N)}{g(N)} = N,$$

which is unbounded. Hence $f(N) \neq O(g(N))$. On the other hand, for even N,

$$\frac{g(N)}{f(N)} = N,$$

which is also unbounded. Hence $g(N) \neq O(f(N))$. Therefore, neither function asymptotically dominates the other.

Exercise 2.6

In a recent court case, a judge cited a city for contempt and ordered a fine of \$2 for the first day. Each subsequent day, until the city followed the judge's order, the fine was squared (i.e., the fine progressed as follows: \$2, \$4, \$16, \$256, \$65,536, ...).

a. What would be the fine on day N?

b. How many days would it take for the fine to reach D dollars (a Big-Oh answer will do)?

Solution

Let F_N be the fine on day N. The rule gives

$$F_1 = 2, \qquad F_N = F_{N-1}^2.$$

a. The exponent of 2 doubles each day:

$$F_1 = 2^1, \quad F_2 = 2^2, \quad F_3 = 2^4, \quad F_4 = 2^8.$$

Therefore, the fine on day N is

$$\boxed{F_N = 2^{2^{N-1}}}.$$

b. To find the first day on which the fine is at least D, solve

$$2^{2^{N-1}} \geq D.$$

Taking the base-2 logarithm twice gives

$$N - 1 \geq \log_2 \log_2 D.$$

Thus, for $D \geq 2$, the first such day is

$$\boxed{N = 1 + \lceil \log_2 \log_2 D \rceil},$$

and the number of days is $\Theta(\log \log D)$.

Exercise 2.7

For each of the following six program fragments:

a. Give an analysis of the running time (Big-Oh will do).

b. Implement the code in the language of your choice, and give the running time for several values of N.

c. Compare your analysis with the actual running times.

```
// (1)
sum = 0;
for (i = 0; i < n; ++i)
    ++sum;

// (2)
sum = 0;
for (i = 0; i < n; ++i)
    for (j = 0; j < n; ++j)
        ++sum;

// (3)
sum = 0;
for (i = 0; i < n; ++i)
    for (j = 0; j < n * n; ++j)
        ++sum;

// (4)
sum = 0;
for (i = 0; i < n; ++i)
    for (j = 0; j < i; ++j)
        ++sum;

// (5)
sum = 0;
for (i = 0; i < n; ++i)
    for (j = 0; j < i * i; ++j)
        for (k = 0; k < j; ++k)
            ++sum;

// (6)
sum = 0;
for (i = 1; i < n; ++i)
    for (j = 1; j < i * i; ++j)
        if (j % i == 0)
            for (k = 0; k < j; ++k)
                ++sum;
```

Solution

Running-time analysis. Counting executions of the innermost statement gives:

(1) The loop executes N times, so the running time is $\Theta(N)$.

(2) Each of the two loops executes N times, giving N^2 increments and a running time of $\Theta(N^2)$.

(3) The inner loop executes N^2 times for each of N outer iterations. Thus the running time is $\Theta(N^3)$.

(4) The number of increments is

$$\sum_{i=0}^{N-1} i = \frac{N(N-1)}{2},$$

so the running time is $\Theta(N^2)$.

(5) For a fixed i, the innermost statement executes

$$\sum_{j=0}^{i^2-1} j = \frac{i^2(i^2-1)}{2} = \Theta(i^4)$$

times. Summing over i gives $\Theta(\sum_{i=0}^{N-1} i^4) = \Theta(N^5)$.

(6) For a fixed i, the condition succeeds when $j = mi$ for $m = 1, \ldots, i-1$. The innermost statement therefore executes

$$\sum_{m=1}^{i-1} mi = \frac{i^2(i-1)}{2} = \Theta(i^3)$$

times. Summing over i gives $\Theta(N^4)$. The $\Theta(N^3)$ condition checks do not change this bound.

Hence the six bounds are

$$\boxed{\Theta(N),\ \Theta(N^2),\ \Theta(N^3),\ \Theta(N^2),\ \Theta(N^5),\ \Theta(N^4)}.$$

C++ implementation. The following functions were timed separately. A volatile accumulator is used so that an optimizing compiler cannot remove the loops.

```cpp
#include <cstdint>

volatile std::uint64_t sum;

void fragment1(int n) {
    sum = 0;
    for (int i = 0; i < n; ++i) ++sum;
}

void fragment2(int n) {
    sum = 0;
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < n; ++j) ++sum;
}

void fragment3(int n) {
    sum = 0;
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < n * n; ++j) ++sum;
}

void fragment4(int n) {
    sum = 0;
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < i; ++j) ++sum;
}

void fragment5(int n) {
    sum = 0;
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < i * i; ++j)
            for (int k = 0; k < j; ++k) ++sum;
}

void fragment6(int n) {
    sum = 0;
    for (int i = 1; i < n; ++i)
        for (int j = 1; j < i * i; ++j)
            if (j % i == 0)
                for (int k = 0; k < j; ++k) ++sum;
}
```

Representative timings. The benchmark was compiled with Microsoft Visual C++ in Release x64 mode and run on an Intel Core i7-13700HX. The following filtered measurements are in microseconds. Absolute times depend on the machine; the scaling ratios are the important part.

N	**F1**	**F2**	**F3**	**F4**	**F5**	**F6**
10	0.0073	0.1450	1.0190	0.0438	14.4	2.1
20	0.0117	0.1256	3.2540	0.3083	174.9	15.1
30	0.0161	0.3765	7.1690	0.4341	640.9	79.7
40	0.0190	0.8298	14.401	0.7059	3,782.3	113.6
50	0.0261	0.6919	26.805	0.8927	8,979.9	224.6

Doubling-ratio comparison. For a running time of order N^k, doubling N should multiply the time by approximately 2^k. Selected stable measurements from the scaling benchmark are shown below.

Fragment	**Complexity**	$N \to 2N$	**Time (μs)**	**Observed**	**Expected**
1	$\Theta(N)$	200k→400k	41.70→84.00	2.01	2
2	$\Theta(N^2)$	4k→8k	3,385.1→13,556.1	4.00	4
3	$\Theta(N^3)$	400→800	13,819.2→108,560.0	7.86	8
4	$\Theta(N^2)$	4k→8k	1,728.5→6,988.2	4.04	4
5	$\Theta(N^5)$	40→80	3,034.5→84,129.0	27.72	32
6	$\Theta(N^4)$	40→80	119.3→1,936.3	16.23	16

Fragments (1)–(4) and (6) closely match their theoretical doubling factors. Fragment (5) is somewhat below the ideal factor of 32 at the tested sizes, but its much steeper scaling is consistent with $\Theta(N^5)$. Timer resolution, caching, processor scheduling, and compiler effects explain the remaining variation. Overall, the i7-13700HX measurements support all six asymptotic bounds.

Exercise 2.8

Suppose that we need to generate a random permutation of the first N integers. For example, $\{4, 3, 1, 5, 2\}$ and $\{3, 1, 4, 2, 5\}$ are legal permutations, whereas $\{5, 4, 1, 2, 1\}$ is not. Assume that `randInt(i,j)` generates every integer between i and j with equal probability. Consider the following algorithms.

1. Fill $a[0]$ through $a[N-1]$. To fill $a[i]$, generate random integers until one is obtained that does not already occur in $a[0], \ldots, a[i-1]$.

2. Proceed as in algorithm (1), but maintain a Boolean array `used`. When a value is inserted, mark it as used, allowing membership to be tested in constant time.

3. First set $a[i] = i + 1$, and then execute

```
for (int i = 1; i < n; ++i)
    swap(a[i], a[randInt(0, i)]);
```

a. Prove that all three algorithms produce only legal permutations and that all permutations are equally likely.

b. Give as accurate a Big-Oh analysis as possible for the expected running time of each algorithm.

c. Implement each algorithm, run it ten times for the requested values of N, and compute the average running times.

d. Compare the analysis with the measured running times.

e. Give the worst-case running time of each algorithm.

Solution

(a) Correctness and uniformity. Algorithms (1) and (2) accept only values not already present in the array, so their outputs are legal permutations. When i positions have been filled, rejection sampling makes each of the $N - i$ unused values equally likely to be selected next. Consequently, every particular permutation has probability

$$\frac{1}{N} \cdot \frac{1}{N-1} \cdots \frac{1}{1} = \frac{1}{N!}.$$

Algorithm (3) is the forward Fisher–Yates shuffle. After iteration i, the prefix $a[0], \ldots, a[i]$ is a uniformly random permutation of its $i + 1$ values. This is true initially, and choosing the swap position uniformly from 0 through i preserves the claim inductively. After the final iteration, every one of the $N!$ permutations therefore has probability $1/N!$.

(b) Expected running times. For algorithms (1) and (2), when i positions have been filled, a random draw succeeds with probability $(N - i)/N$. The expected number of draws is therefore

$$\frac{N}{N-i}.$$

In algorithm (1), a successful membership search examines all i existing entries. The expected number of failed draws is $i/(N-i)$, and a failed search examines $(i+1)/2$ entries on average. Thus the expected work at position i is

$$i + \frac{i}{N-i}\frac{i+1}{2}.$$

Summing this expression gives

$$\boxed{\mathbb{E}[T_1(N)] = \Theta(N^2 \log N)}.$$

Algorithm (2) tests `used` in constant time, so its expected number of random draws is

$$\sum_{i=0}^{N-1} \frac{N}{N-i} = N \sum_{r=1}^{N} \frac{1}{r} = N H_N.$$

Therefore,

$$\boxed{\mathbb{E}[T_2(N)] = \Theta(N \log N)}.$$

Algorithm (3) performs N initial assignments and exactly $N-1$ swaps, giving

$$\boxed{T_3(N) = \Theta(N)}.$$

(c) C++ implementation. The following implementations use the standard Mersenne Twister random number generator.

```
#include <algorithm>
#include <numeric>
#include <random>
#include <vector>

std::vector<int> permutation1(int n, std::mt19937& rng) {
    std::vector<int> a(n);
    std::uniform_int_distribution<int> pick(1, n);
    for (int i = 0; i < n; ++i) {
        while (true) {
            int value = pick(rng);
            bool duplicate = false;
            for (int j = 0; j < i; ++j)
                if (a[j] == value) {
                    duplicate = true;
                    break;
                }
            if (!duplicate) {
                a[i] = value;
                break;
            }
        }
    }
    return a;
}

std::vector<int> permutation2(int n, std::mt19937& rng) {
    std::vector<int> a(n);
    std::vector<bool> used(n + 1, false);
    std::uniform_int_distribution<int> pick(1, n);
    for (int i = 0; i < n; ++i) {
        int value;
        do {
            value = pick(rng);
        } while (used[value]);
        a[i] = value;
        used[value] = true;
    }
    return a;
}

std::vector<int> permutation3(int n, std::mt19937& rng) {
    std::vector<int> a(n);
    std::iota(a.begin(), a.end(), 1);
    for (int i = 1; i < n; ++i) {
        std::uniform_int_distribution<int> pick(0, i);
        std::swap(a[i], a[pick(rng)]);
    }
    return a;
}
```

Each function was executed ten times for each input size. The following table reports the average elapsed time in milliseconds from a representative optimized C++17 run.

Algorithm	N	Average time (ms)
1	250	0.069
1	500	0.307
1	1,000	1.198
1	2,000	4.380
2	25,000	1.501
2	50,000	2.845
2	100,000	6.255
2	200,000	13.439
2	400,000	28.003
2	800,000	67.784
3	100,000	0.494
3	200,000	1.035
3	400,000	2.078
3	800,000	4.441
3	1,600,000	10.133
3	3,200,000	24.167
3	6,400,000	76.557

(d) Comparison. Algorithm (1) becomes expensive at only a few thousand elements, and its time increases by roughly a factor of four to five when N doubles, consistent with $\Theta(N^2 \log N)$. Algorithm (2) grows a little more than linearly when N doubles, as predicted by $\Theta(N \log N)$. Algorithm (3) shows approximately linear scaling until memory-allocation and cache effects become significant at the largest inputs. The measurements therefore agree with the expected-time analysis.

(e) Worst-case running times. Algorithms (1) and (2) use rejection sampling. An unlucky random-number sequence can repeatedly generate an already-used value indefinitely, so neither algorithm has a finite worst-case running-time bound. Their worst-case running time is unbounded. Algorithm (3) always performs the same number of assignments and swaps, so its worst-case time is $\Theta(N)$.

Exercise 2.9

Complete the table in Figure 2.2 with estimates for the running times that were too long to simulate. Interpolate the running times for these algorithms and estimate the time required to compute the maximum subsequence sum of one million numbers. What assumptions have you made?

Solution

For a tenfold increase in input size, the expected multiplication factors are

$$10^3 = 1000, \qquad 10^2 = 100, \qquad 10\left(1 + \frac{\log 10}{\log N}\right), \qquad 10$$

for Algorithms 1 through 4, respectively. Applying these factors to the most recent available measurements gives the completed table below. Times are in seconds; the entries marked with an asterisk are estimates.

Input size	**Alg. 1** $O(N^3)$	**Alg. 2** $O(N^2)$	**Alg. 3** $O(N \log N)$	**Alg. 4** $O(N)$
100	0.000159	0.000006	0.000005	0.000002
1,000	0.095857	0.000371	0.000060	0.000022
10,000	86.67	0.033322	0.000619	0.000222
100,000	86,670*	3.33	0.006700	0.002205
1,000,000	86,670,000*	333*	0.074870	0.022711

For one million numbers, the estimated running times are therefore

Algorithm	Seconds	More readable form
1	86,670,000	about 1,003 days, or 2.75 years
2	333	about 5.55 minutes
3	0.074870	about 74.9 ms
4	0.022711	about 22.7 ms

As a check on the interpolation, Algorithm 3 predicted from the $N = 100{,}000$ row would take

$$0.006700\left(10\frac{\log(1{,}000{,}000)}{\log(100{,}000)}\right) = 0.006700(12) = 0.0804 \text{ seconds},$$

which is close to the measured 0.074870 seconds. Likewise, the linear prediction for Algorithm 4 is $10(0.002205) = 0.02205$ seconds, close to the measured 0.022711 seconds.

The estimates assume that the dominant asymptotic term already controls the measured running time, its constant factor remains stable, and the same hardware, compiler, optimization settings, and input conditions are used. They also ignore secondary terms and effects such as caching, memory bandwidth, operating-system scheduling, and timer error.

Exercise 2.10

Determine, for the typical algorithms used to perform calculations by hand, the running time required to do the following:

a. Add two N-digit integers.

b. Multiply two N-digit integers.

c. Divide two N-digit integers.

Solution

Assume that a single-digit arithmetic operation, comparison, or carry operation takes constant time.

a. In ordinary addition, each pair of corresponding digits is processed once, together with a possible carry. Thus the running time is

$$\boxed{\Theta(N)}.$$

b. In grade-school multiplication, each of the N digits of one number is multiplied by each of the N digits of the other. Adding the shifted partial products also takes at most quadratic time. Therefore, the running time is

$$\boxed{\Theta(N^2)}.$$

c. Long division determines at most N quotient digits. At each stage, comparing with and subtracting a suitably shifted divisor can require $\Theta(N)$ digit operations. Hence the running time is

$$\boxed{\Theta(N^2)}.$$

These bounds describe the usual hand algorithms; faster multiplication and division algorithms exist for sufficiently large inputs.

Exercise 2.11

An algorithm takes 0.5 ms for input size 100. How long will it take for input size 500 if the running time is the following (assume low-order terms are negligible)?

a. Linear

b. $O(N \log N)$

c. Quadratic

d. Cubic

Solution

The input size increases by a factor of

$$\frac{500}{100} = 5.$$

Scale the original 0.5 ms running time according to each growth rate.

a. For linear time, the multiplication factor is 5:

$$0.5(5) = \boxed{2.5 \text{ ms}}.$$

b. For $N \log N$, the multiplication factor is

$$\frac{500 \log 500}{100 \log 100} = 5\frac{\log 500}{\log 100} \approx 6.747.$$

Therefore, the estimated time is

$$0.5(6.747) \approx \boxed{3.37 \text{ ms}}.$$

c. For quadratic time, the multiplication factor is $5^2 = 25$:

$$0.5(25) = \boxed{12.5 \text{ ms}}.$$

d. For cubic time, the multiplication factor is $5^3 = 125$:

$$0.5(125) = \boxed{62.5 \text{ ms}}.$$

Exercise 2.12

An algorithm takes 0.5 ms for input size 100. How large a problem can be solved in 1 min if the running time is the following (assume low-order terms are negligible)?

a. Linear

b. $O(N \log N)$

c. Quadratic

d. Cubic

Solution

One minute is 60,000 ms, so the available time is larger by the factor

$$R = \frac{60{,}000}{0.5} = 120{,}000.$$

If $T(N) = cf(N)$, we therefore solve

$$\frac{f(N)}{f(100)} = 120{,}000.$$

a. For linear time,

$$\frac{N}{100} = 120{,}000, \qquad \boxed{N = 12{,}000{,}000}.$$

b. For $N \log N$ time, N satisfies

$$\frac{N \log N}{100 \log 100} = 120{,}000.$$

Solving this equation numerically gives

$$N \approx 3{,}656{,}807,$$

so the largest integral input size is approximately

$$\boxed{N \approx 3.66 \times 10^6}.$$

c. For quadratic time,

$$\left(\frac{N}{100}\right)^2 = 120{,}000, \qquad N = 100\sqrt{120{,}000} \approx 34{,}641.$$

Thus,

$$\boxed{N \approx 34{,}641}.$$

d. For cubic time,

$$\left(\frac{N}{100}\right)^3 = 120{,}000, \qquad N = 100\sqrt[3]{120{,}000} \approx 4{,}932.$$

Thus,

$$\boxed{N \approx 4{,}932}.$$

Exercise 2.13

How much time is required to compute $f(x) = \sum_{i=0}^{N} a_i x^i$?

a. Using a simple routine to perform exponentiation?

b. Using the routine in Section 2.4.4?

Solution

Assume that each power x^i is computed independently and that each arithmetic operation takes constant time.

a. A simple exponentiation routine computes x^i using $i - 1 = \Theta(i)$ multiplications. Thus the total work is

$$\sum_{i=0}^{N} \Theta(i) = \Theta(N^2).$$

The additional $N + 1$ coefficient multiplications and additions do not change the bound, so the running time is

$$\boxed{\Theta(N^2)}.$$

b. The fast exponentiation routine from Section 2.4.4 computes x^i in $\Theta(\log i)$ time. Computing all powers independently therefore requires

$$\sum_{i=1}^{N} \Theta(\log i) = \Theta(\log(N!)) = \Theta(N \log N).$$

Hence the total running time is

$$\boxed{\Theta(N \log N)}.$$

Exercise 2.14

Consider the following algorithm (known as Horner's rule) to evaluate $f(x) = \sum_{i=0}^{N} a_i x^i$:

```
poly = 0;
for (i = n; i >= 0; --i)
    poly = x * poly + a[i];
```

a. Show how the steps are performed by this algorithm for $x = 3$, $f(x) = 4x^4+8x^3+x+2$.

b. Explain why this algorithm works.

c. What is the running time of this algorithm?

Solution

The coefficients in ascending order are

$$a_0 = 2, \quad a_1 = 1, \quad a_2 = 0, \quad a_3 = 8, \quad a_4 = 4.$$

a. Starting with `poly = 0`, the iterations give

i	`poly` $= 3($`poly`$) + a_i$
4	$3(0) + 4 = 4$
3	$3(4) + 8 = 20$
2	$3(20) + 0 = 60$
1	$3(60) + 1 = 181$
0	$3(181) + 2 = 545$

Therefore,

$$\boxed{f(3) = 545}.$$

b. The polynomial can be written in nested form as

$$f(x) = a_0 + x\Big(a_1 + x\Big(a_2 + \cdots + x(a_{N-1} + xa_N)\cdots\Big)\Big).$$

After processing coefficient a_i, the variable `poly` contains

$$a_i + a_{i+1}x + a_{i+2}x^2 + \cdots + a_N x^{N-i}.$$

The assignment `poly = x * poly + a[i]` constructs this expression one coefficient at a time, so after the final iteration it equals the original polynomial $f(x)$.

c. The loop executes exactly $N + 1$ times, with one multiplication and one addition per iteration. Hence its running time is

$$\boxed{\Theta(N)}.$$

Exercise 2.15

Give an efficient algorithm to determine if there exists an integer i such that $A_i = i$ in an array of integers $A_1 < A_2 < A_3 < \cdots < A_N$. What is the running time of your algorithm?

Solution

Use binary search on the index. With the array indexed from 1 through N, the algorithm is:

```
bool hasFixedPoint(const vector<int>& a) {
    int low = 1;
    int high = static_cast<int>(a.size()) - 1;

    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (a[mid] == mid)
            return true;
        if (a[mid] < mid)
            low = mid + 1;
        else
            high = mid - 1;
    }
    return false;
}
```

To justify the discarded halves, suppose first that $A_{mid} < mid$. Because the entries are strictly increasing integers, for every $j < mid$,

$$A_j \le A_{mid} - (mid - j) < j.$$

Thus no index to the left can satisfy $A_j = j$. Similarly, if $A_{mid} > mid$, then for every $j > mid$,

$$A_j \ge A_{mid} + (j - mid) > j,$$

so no index to the right can be a solution.

Each comparison eliminates at least half of the remaining indices. Therefore, the running time is

$$\boxed{\Theta(\log N)},$$

with $\Theta(1)$ auxiliary space for the iterative implementation.

Exercise 2.16

Write an alternative gcd algorithm based on the following observations (arrange so that $a > b$):

a. $\gcd(a, b) = 2\gcd(a/2, b/2)$ if a and b are both even.

b. $\gcd(a, b) = \gcd(a/2, b)$ if a is even and b is odd.

c. $\gcd(a, b) = \gcd(a, b/2)$ if a is odd and b is even.

d. $\gcd(a, b) = \gcd((a + b)/2, (a - b)/2)$ if a and b are both odd.

Solution

The following recursive implementation is a form of the binary gcd algorithm. The expression used for $(a + b)/2$ avoids overflow from forming $a + b$ directly.

```
#include <cstdint>
#include <utility>

std::uint64_t binaryGcd(std::uint64_t a, std::uint64_t b) {
    if (a == 0) return b;
    if (b == 0) return a;
    if (a < b) std::swap(a, b);
    if (a == b) return a;

    bool aEven = (a % 2 == 0);
    bool bEven = (b % 2 == 0);

    if (aEven && bEven)
        return 2 * binaryGcd(a / 2, b / 2);
    if (aEven)
        return binaryGcd(a / 2, b);
    if (bEven)
        return binaryGcd(a, b / 2);

    // Both are odd and a > b.
    return binaryGcd(b + (a - b) / 2, (a - b) / 2);
}
```

The first identity removes a common factor of 2. In the second and third cases, an odd number cannot share a factor of 2 with the even number, so dividing the even argument by 2 does not change the gcd.

For the final case, define

$$u = \frac{a+b}{2}, \qquad v = \frac{a-b}{2}.$$

Then $a = u + v$ and $b = u - v$. Consequently, every common divisor of u and v divides both a and b. Conversely, since a and b are odd, their gcd is odd and divides both $(a+b)/2$ and $(a-b)/2$. Therefore $\gcd(a, b) = \gcd(u, v)$.

Each call either halves an even argument or replaces two odd arguments by values formed from their half-sum and half-difference. After at most a constant number of calls, the magnitude of the arguments decreases by a constant factor. Under the usual unit-cost arithmetic model, the worst-case running time is therefore

$$\boxed{\Theta(\log \max(a, b))}.$$

The recursion also uses $\Theta(\log \max(a, b))$ stack space; an iterative version can use constant auxiliary space.

Exercise 2.17

Give efficient algorithms (along with running-time analyses) to:

a. Find the minimum subsequence sum.

b. Find the minimum positive subsequence sum.

c. Find the maximum subsequence product.

Solution

Here, a subsequence means a contiguous subsequence.

a. This is the minimum-sum version of Kadane's algorithm. If the empty subsequence is permitted, initialize both values to zero.

```
long long minimumSubsequenceSum(const vector<int>& a) {
    long long endingHere = 0;
    long long minimumSum = 0;

    for (int x : a) {
        endingHere = std::min(0LL, endingHere + x);
        minimumSum = std::min(minimumSum, endingHere);
    }
    return minimumSum;
}
```

Each element is processed once, so the running time is $\boxed{\Theta(N)}$ and the auxiliary space is $\Theta(1)$.

b. Let the prefix sums be

$$P_0 = 0, \qquad P_j = \sum_{k=1}^{j} A_k.$$

A subsequence from $i+1$ through j has sum $P_j - P_i$. For each P_j, the smallest positive difference is obtained by choosing the largest earlier prefix sum that is strictly less than P_j. Maintain all earlier prefix sums in an ordered set and perform a predecessor query for every P_j.

```
long long minimumPositiveSubsequenceSum(const vector<int>& a) {
    std::set<long long> earlier{0};
    long long prefix = 0;
    long long answer = std::numeric_limits<long long>::max();

    for (int x : a) {
        prefix += x;
        auto it = earlier.lower_bound(prefix);
        if (it != earlier.begin()) {
            --it;
            answer = std::min(answer, prefix - *it);
        }
        earlier.insert(prefix);
    }
    return answer; // No answer if this remains LLONG_MAX.
}
```

Each ordered-set operation takes $O(\log N)$ time, so the total is $\boxed{O(N \log N)}$ time and $O(N)$ space.

c. A negative value can turn the smallest product ending at the previous position into the largest. Therefore, keep both the maximum and minimum products ending at the current position.

```
long long maximumSubsequenceProduct(const vector<int>& a) {
    long long maximumEnding = a[0];
    long long minimumEnding = a[0];
    long long answer = a[0];

    for (std::size_t i = 1; i < a.size(); ++i) {
        long long x = a[i];
        if (x < 0)
            std::swap(maximumEnding, minimumEnding);

        maximumEnding = std::max(x, x * maximumEnding);
        minimumEnding = std::min(x, x * minimumEnding);
        answer = std::max(answer, maximumEnding);
    }
    return answer;
}
```

This algorithm takes $\boxed{\Theta(N)}$ time and $\Theta(1)$ auxiliary space. The numeric type must be large enough to avoid overflow.

Exercise 2.18

An important problem in numerical analysis is to find a solution to $f(x) = 0$ for an arbitrary continuous function f. If $f(low)$ and $f(high)$ have opposite signs, write a function that uses binary search to solve for a zero. What must be done to ensure termination?

Solution

The bisection method repeatedly keeps the half-interval whose endpoint values have opposite signs.

```
double findZero(const std::function<double(double)>& f,
                double low, double high,
                double xTolerance = 1e-12,
                double fTolerance = 1e-12) {
    double fLow = f(low);
    double fHigh = f(high);

    if (std::abs(fLow) <= fTolerance) return low;
    if (std::abs(fHigh) <= fTolerance) return high;
    if (std::signbit(fLow) == std::signbit(fHigh))
        throw std::invalid_argument("root is not bracketed");

    for (int iteration = 0; iteration < 200; ++iteration) {
        double mid = low + (high - low) / 2.0;
        double fMid = f(mid);

        if (std::abs(fMid) <= fTolerance ||
            high - low <= xTolerance ||
            mid == low || mid == high)
            return mid;

        if (std::signbit(fLow) != std::signbit(fMid)) {
            high = mid;
        } else {
            low = mid;
            fLow = fMid;
        }
    }
    return low + (high - low) / 2.0;
}
```

Testing only whether $f(mid) = 0$ does not guarantee termination, because a floating-point computation may never produce the exact root. Termination must instead be based on a tolerance for the interval or function value, together with a maximum iteration count. After k iterations the interval width is $(high - low)/2^k$, so an x-accuracy of ε requires

$$\boxed{O\left(\log \frac{high - low}{\varepsilon}\right)}$$

function evaluations.

Exercise 2.19

Modify the maximum contiguous subsequence sum algorithms so that they return, in a single object, both the maximum sum and the indices of the actual subsequence.

Solution

Store the sum and its two endpoints in a result object. The linear-time algorithm becomes:

```
struct SubsequenceResult {
    long long sum;
    int first;
    int last;
};

SubsequenceResult maximumSubsequence(const vector<int>& a) {
    SubsequenceResult best{0, -1, -1};
    long long currentSum = 0;
    int currentFirst = 0;

    for (int i = 0; i < static_cast<int>(a.size()); ++i) {
        currentSum += a[i];

        if (currentSum > best.sum)
            best = {currentSum, currentFirst, i};

        if (currentSum < 0) {
            currentSum = 0;
            currentFirst = i + 1;
        }
    }
    return best;
}
```

The indices $(-1, -1)$ represent the empty subsequence when all entries are negative. The running time remains $\boxed{\Theta(N)}$, since only constant additional work is performed per element.

The same idea applies to the other algorithms in the text. In the cubic and quadratic versions, record the current i and j whenever a new maximum is found. In the divide-and-conquer version, each recursive call returns a `SubsequenceResult`; the left, right, and crossing results are constructed with their endpoints, and the one with the largest sum is returned. These additions do not change their respective asymptotic running times.

Exercise 2.20

Consider a program that determines whether a positive integer N is prime.

a. Write the program.

b. Give its worst-case running time in terms of N.

c. Let B be the number of bits in the binary representation of N. What is B?

d. Give the worst-case running time in terms of B.

e. Compare testing a 20-bit number and a 40-bit number.

f. Is it more reasonable to state the running time in terms of N or B? Why?

Solution

a. It is sufficient to test possible divisors through $\sqrt{N}$. The comparison $d \leq N/d$ avoids overflow from computing d^2.

```
bool isPrime(std::uint64_t n) {
    if (n < 2) return false;
    if (n % 2 == 0) return n == 2;

    for (std::uint64_t d = 3; d <= n / d; d += 2)
        if (n % d == 0)
            return false;
    return true;
}
```

If N is composite, it has a factor no larger than $\sqrt{N}$; if no such factor is found, N is prime.

b. In the worst case, such as when N is prime, the program tests $\Theta(\sqrt{N})$ possible divisors. Under the unit-cost arithmetic assumption, its running time is

$$\boxed{\Theta(\sqrt{N})}.$$

c. For $N \geq 1$, the binary input length is

$$\boxed{B = \lfloor \log_2 N \rfloor + 1}.$$

d. Since N can be approximately 2^B,

$$\sqrt{N} = \Theta(2^{B/2}).$$

Thus the unit-cost running time in terms of the input length is

$$\boxed{\Theta(2^{B/2})},$$

which is exponential in B. Accounting for the bit cost of division would add a polynomial factor in B.

e. A 20-bit number has square root on the order of 2^{10}, whereas a 40-bit number has square root on the order of 2^{20}. Therefore the 40-bit test can require approximately

$$\frac{2^{20}}{2^{10}} = \boxed{1024}$$

times as many trial divisions.

f. It is more informative to express the running time in terms of B, because B is the actual size of the input representation. The bound $\Theta(\sqrt{N})$ can appear moderate, while the equivalent $\Theta(2^{B/2})$ correctly reveals that trial division is exponential in the number of input bits.

Exercise 2.21

The Sieve of Eratosthenes computes all primes less than N by crossing out multiples of each uncrossed integer through $\sqrt{N}$. What is its running time?

Solution

Initializing and scanning the Boolean table takes $\Theta(N)$ time. For each prime $p \le \sqrt{N}$, crossing out its multiples performs at most N/p operations. Hence the total marking work is

$$\sum_{\substack{p \le \sqrt{N} \\ p \text{ prime}}} \frac{N}{p} = N \sum_{p \le \sqrt{N}} \frac{1}{p}.$$

The sum of the reciprocals of the primes up to x is $\Theta(\log \log x)$, so

$$N \sum_{p \le \sqrt{N}} \frac{1}{p} = \Theta(N \log \log N).$$

The linear initialization and scan are dominated by this term. Therefore, assuming constant-time table operations, the running time is

$$\boxed{\Theta(N \log \log N)},$$

and the table requires $\boxed{\Theta(N)}$ space.

Exercise 2.22

Show that X^{62} can be computed with only eight multiplications.

Solution

Use the following addition chain for the exponents:

$$1,\ 2,\ 3,\ 5,\ 10,\ 20,\ 30,\ 31,\ 62.$$

It gives the computation

$$\begin{aligned} X^2 &= X \cdot X, & X^3 &= X^2 \cdot X, \\ X^5 &= X^3 \cdot X^2, & X^{10} &= X^5 \cdot X^5, \\ X^{20} &= X^{10} \cdot X^{10}, & X^{30} &= X^{20} \cdot X^{10}, \\ X^{31} &= X^{30} \cdot X, & X^{62} &= X^{31} \cdot X^{31}. \end{aligned}$$

Each line contains one multiplication, for a total of exactly

$$\boxed{\text{8 multiplications}}.$$

Exercise 2.23

Write the fast exponentiation routine without recursion.

Solution

The exponent can be processed from its most significant binary digit to its least significant digit. The leading digit is always 1, so the initial value is x.

```
long long power(long long x, unsigned int n) {
    if (n == 0)
        return 1;

    unsigned int mask = 1;
    while (mask <= n / 2)
        mask <<= 1;              // Highest power of two <= n.

    long long result = x;        // Process the leading 1 bit.
    mask >>= 1;

    while (mask != 0) {
        result *= result;
        if ((n & mask) != 0)
            result *= x;
        mask >>= 1;
    }
    return result;
}
```

Squaring shifts the already processed binary exponent one position to the left; multiplying by x appends a 1 bit. Since the number of bits in n is $\Theta(\log n)$, the running time is

$$\boxed{\Theta(\log n)}.$$

Exercise 2.24

Give a precise count of the number of multiplications used by the fast exponentiation routine.

Solution

Let

$$n = (b_k b_{k-1} \cdots b_1 b_0)_2, \qquad b_k = 1,$$

and let $s(n)$ denote the number of 1 bits in this binary representation. There are

$$k = \lfloor \log_2 n \rfloor$$

bits after the leading bit. Processing each of them requires one squaring, giving k multiplications. Every remaining 1 bit also requires one multiplication by x, giving $s(n) - 1$ additional multiplications. Thus, for $n \geq 1$, the precise count is

$$\boxed{M(n) = \lfloor \log_2 n \rfloor + s(n) - 1}.$$

For $n = 0$, no multiplication is required. Since $1 \leq s(n) \leq \lfloor \log_2 n \rfloor + 1$, this count lies between

$$\lfloor \log_2 n \rfloor \quad \text{and} \quad 2\lfloor \log_2 n \rfloor,$$

confirming the $\Theta(\log n)$ bound. For example, $62 = (111110)_2$, so this routine uses $5 + 5 - 1 = 9$ multiplications; the specially chosen addition chain in Exercise 2.22 improves this to eight.

Exercise 2.25

Programs A and B have worst-case running times no greater than $150N \log_2 N$ and N^2, respectively. Answer the following questions, if possible.

a. Which program has the better guarantee for large values ($N > 10{,}000$)?

b. Which program has the better guarantee for small values ($N < 100$)?

c. Which program will run faster on average for $N = 1{,}000$?

d. Is it possible that program B runs faster than program A on all possible inputs?

Solution

Compare the two stated upper bounds by cancelling the positive factor N:

$$150N \log_2 N < N^2 \quad \Longleftrightarrow \quad 150 \log_2 N < N.$$

a. At $N = 10{,}000$,

$$150 \log_2(10{,}000) \approx 1{,}993 < 10{,}000,$$

and the linear factor N eventually grows faster than $\log N$. Thus $150N \log_2 N < N^2$ throughout the stated large-input range, so

$$\boxed{\text{Program } A \text{ has the better worst-case guarantee.}}$$

b. For the relevant integers $2 \leq N < 100$, $150 \log_2 N > N$, and hence $N^2 < 150N \log_2 N$. Therefore,

$$\boxed{\text{Program } B \text{ has the better worst-case guarantee.}}$$

c. This cannot be determined. The given information describes only worst-case upper bounds; it provides neither average-case running times nor an input distribution. Thus

$$\boxed{\text{there is insufficient information.}}$$

d. Yes. An upper bound does not state that a program actually takes that much time, nor does it provide a lower bound. Program B could have a smaller actual running time on every input while still obeying its N^2 upper bound. Hence

$$\boxed{\text{yes, this is possible.}}$$

Exercise 2.26

A majority element in an array A of size N is an element that appears more than $N/2$ times. To find a candidate, compare adjacent pairs, place one copy of each equal pair in a second array B, discard unequal pairs, and recursively find a candidate in B. Then scan A to verify whether the candidate is actually a majority.

a. How does the recursion terminate?

b. How is the case where N is odd handled?

c. What is the running time of the algorithm?

d. How can the extra array B be avoided?

e. Write a program to compute the majority element.

Solution

Pair cancellation is valid because discarding two unequal elements removes at most one occurrence of any particular value. Thus, if an element is a majority before the cancellation, it remains the only possible majority candidate afterward.

a. The recursion stops when the current array has zero or one element. An empty array has no candidate, while the sole element of a one-element array is its candidate. The recursion also stops if a pairing step produces an empty array.

b. Let $N = 2k+1$ and temporarily leave the final element z unpaired. Form B from the first $2k$ elements and recursively obtain its candidate, if one exists. There are then at most two candidates that must be checked in the original array: the candidate from B and z.

   To see why, suppose the true majority is x. If $z \neq x$, then all of the more than $N/2$ copies of x occur in the first $2k$ positions, where x is still a majority and must survive as the recursive candidate. If $z = x$, the remaining copies can be exactly half of the first $2k$ entries, so the recursive reduction might discard them all; in that case z supplies the missing candidate.

c. Forming the paired array takes $\Theta(N)$ time and leaves at most $N/2$ elements. Candidate verification is another linear scan, so the recurrence is

$$T(N) = T(N/2) + \Theta(N).$$

   Expanding it gives

$$T(N) = \Theta\left(N + \frac{N}{2} + \frac{N}{4} + \cdots\right) = \boxed{\Theta(N)}.$$

   A linear lower bound is unavoidable because an element near the end can change whether a candidate has a strict majority.

d. The equal-pair representatives can be written into the beginning of A itself. Maintain a write index, scan the current prefix two elements at a time, and copy one member of every equal pair to `A[write++]`. The recursive call then operates only on this shortened prefix. This removes the auxiliary array, although the recursive version still uses $O(\log N)$ stack space.

   An even simpler constant-space implementation uses the equivalent Boyer–Moore cancellation process: keep one candidate and a balance, cancelling each occurrence of a different value against one occurrence of the candidate.

e. The candidate produced by cancellation must be verified because an array need not contain any majority element.

```cpp
#include <optional>
#include <vector>

std::optional<int> majorityElement(const std::vector<int>& a) {
    if (a.empty())
        return std::nullopt;

    int candidate = 0;
    int balance = 0;

    for (int value : a) {
        if (balance == 0) {
            candidate = value;
            balance = 1;
        } else if (value == candidate) {
            ++balance;
        } else {
            --balance;
        }
    }

    std::size_t occurrences = 0;
    for (int value : a)
        if (value == candidate)
            ++occurrences;

    if (occurrences > a.size() / 2)
        return candidate;
    return std::nullopt;
}
```

The first scan finds the only possible candidate and the second scan verifies it. The program therefore uses $\boxed{\Theta(N)}$ time and $\boxed{\Theta(1)}$ auxiliary space.

Exercise 2.27

The input is an N by N matrix already in memory. Every row is increasing from left to right, and every column is increasing from top to bottom. Give an $O(N)$ worst-case algorithm that determines whether a number X occurs in the matrix.

Solution

Begin at the upper-right corner. If the current element is larger than X, move left; if it is smaller than X, move down.

```
bool contains(const vector<vector<int>>& matrix, int x) {
    int n = static_cast<int>(matrix.size());
    int row = 0;
    int column = n - 1;

    while (row < n && column >= 0) {
        if (matrix[row][column] == x)
            return true;
        if (matrix[row][column] > x)
            --column;
        else
            ++row;
    }
    return false;
}
```

If the current value is greater than X, every element below it in the same column is still larger, so that column can be discarded. If the current value is less than X, every element to its left in the same row is still smaller, so that row can be discarded. Thus every move eliminates one complete row or column without discarding a possible match.

There are at most N downward moves and N leftward moves, so the algorithm performs at most $2N - 1$ comparisons. Its worst-case running time is therefore

$$\boxed{\Theta(N)},$$

with $\Theta(1)$ auxiliary space.

Exercise 2.28

Design efficient algorithms that take an array of positive numbers a and determine, subject to $j \geq i$, the maximum value of:

a. $a[j] + a[i]$

b. $a[j] - a[i]$

c. $a[j]a[i]$

d. $a[j]/a[i]$

Solution

For each position j, maintain the minimum and maximum values in the prefix $a[0], \ldots, a[j]$. This ensures that the chosen index i always satisfies $i \leq j$.

```cpp
struct PairExtrema {
    double maximumSum;
    double maximumDifference;
    double maximumProduct;
    double maximumRatio;
};

PairExtrema findExtrema(const vector<double>& a) {
    double prefixMinimum = a[0];
    double prefixMaximum = a[0];

    PairExtrema answer{
        2 * a[0], 0.0, a[0] * a[0], 1.0
    };

    for (double value : a) {
        prefixMinimum = std::min(prefixMinimum, value);
        prefixMaximum = std::max(prefixMaximum, value);

        answer.maximumSum = std::max(
            answer.maximumSum, value + prefixMaximum);
        answer.maximumDifference = std::max(
            answer.maximumDifference, value - prefixMinimum);
        answer.maximumProduct = std::max(
            answer.maximumProduct, value * prefixMaximum);
        answer.maximumRatio = std::max(
            answer.maximumRatio, value / prefixMinimum);
    }
    return answer;
}
```

Because $i = j$ is allowed and all values are positive, parts (a) and (c) simplify to

$$\boxed{2 \max_k a[k]} \qquad \text{and} \qquad \boxed{\left(\max_k a[k]\right)^2},$$

respectively. For each fixed j, the best choice in part (b) is the smallest earlier value, giving

$$\boxed{\max_j \left(a[j] - \min_{i \le j} a[i]\right)},$$

and positivity makes the same prefix minimum the best denominator in part (d):

$$\boxed{\max_j \left(\frac{a[j]}{\min_{i \le j} a[i]}\right)}.$$

All four answers are obtained in one pass, so the running time is $\boxed{\Theta(N)}$ and the auxiliary space is $\boxed{\Theta(1)}$.

Exercise 2.29

Why is it important to assume that integers in our computer model have a fixed size?

Solution

The usual RAM model counts an integer arithmetic operation, comparison, assignment, or memory access as taking constant time. This is realistic only when every integer fits in a fixed-size machine word.

If integers could contain arbitrarily many bits while addition, multiplication, division, and comparison were still charged constant time, a single "integer operation" could process an unbounded amount of information. For example, an entire long input could be encoded in one enormous integer, and the model would incorrectly treat operations on that encoding as taking the same time as operations on a small integer.

With arbitrary-precision integers, the cost must instead depend on the operand length. Addition of two B-bit integers takes $\Theta(B)$ time with the ordinary method, while multiplication and division require still more work. The input size of a numerical value N is also $\Theta(\log N)$ bits rather than N itself.

Therefore, the fixed-size assumption keeps constant-time primitive operations and the resulting running-time analyses meaningful. When values can grow beyond a machine word, their bit lengths must be included explicitly in the analysis.

Exercise 2.30

Consider the word-puzzle problem from Chapter 1. Suppose the length of the longest word is fixed at 10 characters.

a. In terms of R and C, the numbers of rows and columns, and W, the number of words, what are the running times of the algorithms described in Chapter 1?

b. Suppose the word list is presorted. Show how binary search gives an algorithm with significantly better running time.

Solution

There are eight possible directions from any starting cell. Since the maximum word length is the fixed constant 10, checking or constructing one possible word takes $O(1)$ time.

a. The first algorithm considers each of the W dictionary words at every one of the RC starting cells and in each of the eight directions. A comparison examines at most 10 characters. Therefore,

$$T_1(R, C, W) = O(8 \cdot 10 \cdot RCW) = \boxed{O(RCW)}.$$

The second algorithm considers each starting cell, each direction, and each possible length from 1 through 10. This produces only $O(8 \cdot 10 \cdot RC) = O(RC)$ candidate strings. Testing a candidate by a sequential scan of the W-word list takes $O(W)$ time, so

$$T_2(R, C, W) = O(RC \cdot W) = \boxed{O(RCW)}.$$

Thus the two straightforward algorithms have the same asymptotic worst-case bound when the maximum word length is fixed, although their constant factors can differ.

b. Use the second algorithm, but look up each generated candidate string with binary search in the sorted word list. A lookup now takes $O(\log W)$ comparisons rather than $O(W)$ comparisons.

```
for each cell (row, column)
    for each of the 8 directions
        word = ""
        for length = 1 to 10
            if the next cell is outside the board
                break
            append the next character to word
            if binarySearch(dictionary, word)
                report word
```

There are $O(RC)$ generated strings because both the number of directions and the maximum length are constants. Consequently, the running time becomes

$$\boxed{O(RC \log W)}.$$

The word list itself can be sorted once in $O(W \log W)$ time if it is not already sorted. If the same dictionary is used for many puzzles, this preprocessing cost is paid only once.

Exercise 2.31

Suppose that the binary-search routine used `low = mid` instead of `low = mid + 1`. Would the routine still work?

Solution

No. When the search interval contains two adjacent positions,

$$high = low + 1,$$

integer division gives

$$mid = \left\lfloor \frac{low + high}{2} \right\rfloor = low.$$

If $a[mid] < x$, the assignment `low = mid` leaves `low`, `mid`, and `high` unchanged. The next iteration repeats exactly the same comparison, so the loop does not terminate.

For example, searching for 4 in $[1, 3]$ first examines the element 1. Setting `low = mid` keeps the interval at $[0, 1]$ forever. The correct assignment is

$$\boxed{\texttt{low = mid + 1}},$$

because the comparison has already proved that position `mid` cannot contain the target.

Exercise 2.32

Implement binary search so that only one two-way comparison is performed in each iteration.

Solution

Maintain the half-open interval $[low, high)$ and find the first position whose value is not less than x. Each iteration asks only whether `a[mid] < x`.

```
template <typename Comparable>
int binarySearch(const vector<Comparable>& a,
                 const Comparable& x) {
    int low = 0;
    int high = static_cast<int>(a.size());

    while (low < high) {
        int mid = low + (high - low) / 2;

        if (a[mid] < x)          // The only comparison in the loop.
            low = mid + 1;
        else
            high = mid;
    }

    if (low < static_cast<int>(a.size()) && a[low] == x)
        return low;
    return -1;
}
```

On termination, `low` is the first index at which an occurrence of x could appear. One final equality test determines whether x is actually present. The running time remains

$$\boxed{\Theta(\log N)}.$$

Exercise 2.33

Suppose that the recursive calls in the divide-and-conquer maximum subsequence sum algorithm are changed to

```
int maxLeftSum  = maxSumRec(a, left, center - 1);
int maxRightSum = maxSumRec(a, center, right);
```

Would the routine still work?

Solution

No. Consider a subproblem containing exactly two elements, so that

$$right = left + 1.$$

With the original midpoint calculation,

$$center = \left\lfloor \frac{left + right}{2} \right\rfloor = left.$$

The proposed right recursive call is then

$$\text{maxSumRec}(a, center, right) = \text{maxSumRec}(a, left, right),$$

which is identical to the current call. The subproblem does not become smaller, so the routine recurses indefinitely and eventually exhausts the call stack.

The original partition

$$[\mathit{left}, \mathit{center}] \quad \text{and} \quad [\mathit{center}+1, \mathit{right}]$$

guarantees that both recursive subproblems are strictly smaller. Thus the proposed replacement

does not work with the original midpoint and base case.

Exercise 2.34

The innermost code of the cubic maximum subsequence sum algorithm runs $N(N+1)(N+2)/6$ times, the quadratic version runs $N(N+1)/2$ times, and the linear version runs N times. What pattern is evident, and what is its combinatorial explanation?

Solution

The three counts are consecutive combinations with repetition:

$$N = \binom{N}{1}, \qquad \frac{N(N+1)}{2} = \binom{N+1}{2}, \qquad \frac{N(N+1)(N+2)}{6} = \binom{N+2}{3}.$$

Thus the evident pattern is

$$\boxed{\binom{N+k-1}{k}}$$

for k nested indices constrained to occur in nondecreasing order.

In the quadratic algorithm, an execution corresponds to a pair of indices

$$0 \le i \le j < N.$$

This is the same as choosing a multiset of two indices from N possible positions, of which there are $\binom{N+1}{2}$.

In the cubic algorithm, an execution of the innermost statement is identified by a triple

$$0 \le i \le k \le j < N,$$

where i and j are the subsequence endpoints and k is the element being added. Such triples are multisets of three indices selected from N positions, so their number is

$$\binom{N+2}{3} = \frac{N(N+1)(N+2)}{6}.$$

The linear algorithm selects only one index, giving $\binom{N}{1} = N$. This multiset interpretation explains all three formulas.